Experiment 2: Finite State Automaton in Python

1. Aim

To write a Python program to implement a basic finite state automaton that recognizes strings ending with `'ab'`.

2. Procedure

1. Define the transition table for states `q0`, `q1`, and `q2`.
2. Set the initial state to `q0` and the final accepting state to `q2`.
3. Traverse each character of the input string and update the current state.
4. Check whether the state after processing the string equals `q2`.
5. Display whether each test string is accepted or rejected.

3. Program

```
transitions = {'q0': {'a': 'q1', 'b': 'q0'}, 'q1': {'a': 'q1', 'b': 'q2'}, 'q2': {'a': 'q1', 'b': 'q0'}}

def accepts(s):
    state = 'q0'
    for ch in s:
        state = transitions[state].get(ch, 'q0')
    return state == 'q2'

for w in ['ab', 'aab', 'aba', 'cab']:
    print(w, "->", "Accepted" if accepts(w) else "Rejected")
```

4. Output

```
ab -> Accepted
aab -> Accepted
aba -> Rejected
cab -> Accepted
```

5. Result

Thus, the Python program was successfully executed to implement a basic finite state automaton that matches strings ending with `'ab'`.

Experiment 3: Morphological Analysis using NLTK in Python

1. Aim

To write a Python program that demonstrates how to perform morphological analysis using the NLTK library.

2. Procedure

1. Import `PorterStemmer` and `WordNetLemmatizer` from the `nltk.stem` module.
2. Initialize the stemmer and lemmatizer objects.
3. Define a list of inflected words for morphological analysis.
4. Compute the stem and base lemma for each word in the list.
5. Display the original word, its root stem, and its morphological lemma.

3. Program

```
from nltk.stem import PorterStemmer, WordNetLemmatizer

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

words = ["running", "played", "studies", "leaves"]

for w in words:
    print(w, "-> Stem:", stemmer.stem(w), "| Lemma:", lemmatizer.lemmatize(w, pos='v'))
```

4. Output

```
running -> Stem: run | Lemma: run
played -> Stem: play | Lemma: play
studies -> Stem: studi | Lemma: study
leaves -> Stem: leav | Lemma: leave
```

5. Result

Thus, the Python program was successfully executed to perform morphological analysis using `PorterStemmer` and `WordNetLemmatizer` in NLTK.

Experiment 4: Finite-State Machine for Morphological Parsing in Python

1. Aim

To write a Python program to implement a finite-state machine for morphological parsing to generate plural forms of English nouns.

2. Procedure

1. Define a morphological parsing function that accepts a singular English noun.
2. Check if the noun ends with sibilant suffixes (`'s'`, `'x'`, `'z'`, `'ch'`, `'sh'`) and append `'es'`.
3. Check if the noun ends with a consonant followed by `'y'` and replace `'y'` with `'ies'`.
4. Append `'s'` for all other regular nouns.
5. Display the singular nouns and their generated plural forms.

3. Program

```
def pluralize(noun):
    if noun.endswith(('s', 'x', 'z', 'ch', 'sh')):
        return noun + 'es'
    elif noun.endswith('y') and noun[-2] not in 'aeiou':
        return noun[:-1] + 'ies'
    return noun + 's'

nouns = ["cat", "bus", "city", "box", "boy"]
for n in nouns:
    print(n, "->", pluralize(n))
```

4. Output

```
cat -> cats
bus -> buses
city -> cities
box -> boxes
boy -> boys
```

5. Result

Thus, the Python program was successfully executed to implement a finite-state machine for morphological parsing to generate plural nouns.

Experiment 5: Word Stemming using Porter Stemmer in Python

1. Aim

To write a Python program to use the Porter Stemmer algorithm to perform word stemming on a list of words.

2. Procedure

1. Import `PorterStemmer` from the `nltk.stem` module.
2. Create an instance of the `PorterStemmer` class.
3. Define a list of words with different morphological suffixes.
4. Apply `ps.stem()` to reduce each word to its root stem.
5. Display the original words and their stemmed outputs.

3. Program

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()
words = ["program", "programs", "programmer", "programming", "programmers"]

stemmed = [ps.stem(w) for w in words]
print("Original Words:", words)
print("Stemmed Words:", stemmed)
```

4. Output

```
Original Words: ['program', 'programs', 'programmer', 'programming',
'programmers']
Stemmed Words: ['program', 'program', 'programm', 'program', 'programm']
```

5. Result

Thus, the Python program was successfully executed to perform word stemming on a list of words using `PorterStemmer`.

Experiment 6: Basic N-Gram Model for Text Generation in Python

1. Aim

To write a Python program to implement a basic N-gram (bigram) model for text generation.

2. Procedure

1. Import `random` and `defaultdict` in Python.
2. Define a sample training text and split it into words.
3. Build a bigram transition dictionary mapping each word to its following words.
4. Start with an initial word and repeatedly sample the next word from the bigram model.
5. Display the bigram dictionary and the generated text sequence.

3. Program

```
import random
from collections import defaultdict

random.seed(1)
words = "the cat sat on the mat the cat ate".split()
bigrams = defaultdict(list)

for w1, w2 in zip(words[:-1], words[1:]):
    bigrams[w1].append(w2)

curr, res = "the", ["the"]
for _ in range(5):
    curr = random.choice(bigrams[curr])
    res.append(curr)

print("Generated Text:", " ".join(res))
```

4. Output

```
Generated Text: the cat sat on the mat
```

5. Result

Thus, the Python program was successfully executed to implement a basic bigram model and generate text.

Experiment 7: Part-of-Speech Tagging using NLTK in Python

1. Aim

To write a Python program using the NLTK library to perform part-of-speech (POS) tagging on a text.

2. Procedure

1. Import the `nltk` module in Python.
2. Define a sample sentence for part-of-speech tagging.
3. Use `nltk.word_tokenize()` to split the text into tokens.
4. Use `nltk.pos_tag()` to assign POS tags to each token.
5. Display the tokenized words and their corresponding POS tags.

3. Program

```
import nltk

text = "Python is a powerful programming language."

tokens = nltk.word_tokenize(text)
print("Tokens:", tokens)

tags = nltk.pos_tag(tokens)
print("POS Tags:", tags)
```

4. Output

```
Tokens: ['Python', 'is', 'a', 'powerful', 'programming', 'language', '.']
POS Tags: [('Python', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('powerful',
'JJ'), ('programming', 'NN'), ('language', 'NN'), ('.', '.')]
```

5. Result

Thus, the Python program was successfully executed to perform part-of-speech tagging on a text using `nltk.pos_tag()`.

Experiment 8: Stochastic Part-of-Speech Tagging in Python

1. Aim

To write a Python program to implement a simple stochastic part-of-speech tagging algorithm using a basic probabilistic model.

2. Procedure

1. Define a tagged training corpus containing word and POS tag pairs.
2. Count the frequency of each tag per word using `defaultdict` and `Counter`.
3. Determine the most probable POS tag for each known word based on maximum likelihood.
4. Assign the default tag `'NN'` for unseen words.
5. Display the stochastically tagged output for a test sentence.

3. Program

```
from collections import defaultdict, Counter

corpus = [("The", "DT"), ("dog", "NN"), ("runs", "VBZ"), ("runs", "VBZ"), ("runs", "NNS")]
counts = defaultdict(Counter)

for word, tag in corpus:
    counts[word][tag] += 1

words = ["The", "dog", "runs", "fast"]
tagged = [(w, counts[w].most_common(1)[0][0] if w in counts else "NN") for w in words]
print("Stochastic POS Tags:", tagged)
```

4. Output

```
Stochastic POS Tags: [('The', 'DT'), ('dog', 'NN'), ('runs', 'VBZ'),
('fast', 'NN')]
```

5. Result

Thus, the Python program was successfully executed to implement a simple stochastic part-of-speech tagging algorithm using a probabilistic model.

Experiment 9: Rule-Based Part-of-Speech Tagging in Python

1. Aim

To write a Python program to implement a rule-based part-of-speech tagging system using regular expressions.

2. Procedure

1. Import `RegexpTagger` from the `nltk.tag` module.
2. Define a list of regular expression rules mapping suffix and number patterns to POS tags.
3. Initialize the `RegexpTagger` with the defined rule patterns.
4. Apply `tagger.tag()` on a tokenized test sentence.
5. Display the rule-based POS tagged output.

3. Program

```
from nltk.tag import RegexpTagger

patterns = [(r'.*ing$', 'VBG'), (r'.*ed$', 'VBD'), (r'.*ly$', 'RB'), (r'.*s$', 'NNS'), (r'^[0-9]+$', 'CD'), (r'.*', 'NN')]

tagger = RegexpTagger(patterns)

words = ["dogs", "barked", "loudly", "running", "3", "cat"]
print("Rule-Based Tags:", tagger.tag(words))
```

4. Output

```
Rule-Based Tags: [('dogs', 'NNS'), ('barked', 'VBD'), ('loudly', 'RB'), ('running', 'VBG'), ('3', 'CD'), ('cat', 'NN')]
```

5. Result

Thus, the Python program was successfully executed to implement a rule-based part-of-speech tagging system using `RegexpTagger`.

Experiment 10: Transformation-Based Part-of-Speech Tagging in Python

1. Aim

To write a Python program to implement transformation-based tagging using a set of transformation rules.

2. Procedure

1. Define a tokenized sentence and its initial baseline POS tags.
2. Define a contextual transformation rule: change `'NN'` to `'VB'` if the previous tag is `'TO'`.
3. Iterate through the tagged words and check if the preceding tag satisfies the rule.
4. Apply the tag transformation where the rule condition is met.
5. Display the initial tags and the final transformed POS tags.

3. Program

```
words = ["I", "want", "to", "race"]
tags = ["PRP", "VBP", "TO", "NN"]

print("Initial Tags:", list(zip(words, tags)))

for i in range(1, len(tags)):
    if tags[i] == "NN" and tags[i-1] == "TO":
        tags[i] = "VB"

print("Transformed Tags:", list(zip(words, tags)))
```

4. Output

```
Initial Tags: [('I', 'PRP'), ('want', 'VBP'), ('to', 'TO'), ('race',
'NN')]
Transformed Tags: [('I', 'PRP'), ('want', 'VBP'), ('to', 'TO'), ('race',
'VB')]
```

5. Result

Thus, the Python program was successfully executed to implement transformation-based tagging by applying a transformation rule.

Experiment 11: Top-Down Parser for Context-Free Grammar in Python

1. Aim

To write a Python program to implement a simple top-down parser for context-free grammars.

2. Procedure

1. Import `CFG` and `RecursiveDescentParser` from the `nltk` library.
2. Define a context-free grammar using `CFG.fromstring()`.
3. Create a top-down parser object using `RecursiveDescentParser`.
4. Split the input sentence into tokens and pass it to `parser.parse()`.
5. Display the generated top-down parse tree.

3. Program

```
import nltk
from nltk import CFG

grammar = CFG.fromstring("S -> NP VP\nNP -> Det N\nVP -> V NP\nDet -> 'the' | 'a'\nN -> 'cat' | 'rat'\nV -> 'chased'")

parser = nltk.RecursiveDescentParser(grammar)
sentence = "the cat chased a rat".split()

for tree in parser.parse(sentence):
    print("Top-Down Parse Tree:", tree)
```

4. Output

```
Top-Down Parse Tree: (S (NP (Det the) (N cat)) (VP (V chased) (NP (Det a) (N rat))))
```

5. Result

Thus, the Python program was successfully executed to implement a top-down parser for context-free grammars using `RecursiveDescentParser`.

Experiment 12: Earley Parser for Context-Free Grammar in Python

1. Aim

To write a Python program to implement an Earley parser for context-free grammars.

2. Procedure

1. Import `CFG` and `EarleyChartParser` from the `nltk` library.
2. Define a context-free grammar using `CFG.fromstring()`.
3. Initialize the `EarleyChartParser` with the grammar.
4. Tokenize the input sentence and parse it using `parser.parse()`.
5. Display the parse tree generated by the Earley parser.

3. Program

```
import nltk
from nltk import CFG

grammar = CFG.fromstring("S -> NP VP\nNP -> Det N | 'John'\nVP -> V NP\nDet -> 'the'\nN -> 'ball'\nV -> 'hit'")

parser = nltk.EarleyChartParser(grammar)
sentence = "John hit the ball".split()

for tree in parser.parse(sentence):
    print("Earley Parse Tree:", tree)
```

4. Output

```
Earley Parse Tree: (S (NP John) (VP (V hit) (NP (Det the) (N ball))))
```

5. Result

Thus, the Python program was successfully executed to implement an Earley parser for context-free grammars using `EarleyChartParser`.

Experiment 13: Parse Tree Generation using CFG in Python

1. Aim

To write a Python program to generate a parse tree for a given sentence using a context-free grammar.

2. Procedure

1. Import `CFG` and `ChartParser` from the `nltk` module.
2. Define the production rules of the context-free grammar using `CFG.fromstring()`.
3. Initialize `ChartParser` with the defined grammar.
4. Parse the tokenized input sentence to generate its syntactic tree.
5. Display the generated parse tree structure.

3. Program

```
import nltk
from nltk import CFG

grammar = CFG.fromstring("S -> NP VP\nNP -> Det N\nVP -> V NP\nDet -> 'the'\nN -> 'boy' | 'book'\nV -> 'reads'")

parser = nltk.ChartParser(grammar)
sentence = "the boy reads the book".split()

for tree in parser.parse(sentence):
    print("Generated Parse Tree:", tree)
```

4. Output

```
Generated Parse Tree: (S (NP (Det the) (N boy)) (VP (V reads) (NP (Det the) (N book))))
```

5. Result

Thus, the Python program was successfully executed to generate a parse tree for a given sentence using a context-free grammar.

Experiment 14: Grammatical Agreement Checking using CFG in Python

1. Aim

To write a Python program to check for grammatical agreement in sentences based on a context-free grammar's rules.

2. Procedure

1. Import `CFG` and `ChartParser` from the `nltk` module.
2. Define CFG production rules enforcing singular (`NP_SG VP_SG`) and plural (`NP_PL VP_PL`) agreement.
3. Initialize the `ChartParser` with the grammar.
4. Parse test sentences and check whether a valid parse tree is produced.
5. Display the grammatical agreement result for each sentence.

3. Program

```
import nltk
from nltk import CFG

grammar = CFG.fromstring("S -> NP_SG VP_SG | NP_PL VP_PL\nNP_SG -> 'dog'\nNP_PL -> 'dogs'\nVP_SG -> 'barks'\nVP_PL -> 'bark'")
parser = nltk.ChartParser(grammar)

for s in ["dog barks", "dogs bark", "dog bark"]:
    valid = len(list(parser.parse(s.split()))) > 0
    print(f"'{s}' ->", "Agreement Valid" if valid else "Agreement Invalid")
```

4. Output

```
'dog barks' -> Agreement Valid
'dogs bark' -> Agreement Valid
'dog bark' -> Agreement Invalid
```

5. Result

Thus, the Python program was successfully executed to check for agreement in sentences based on a context-free grammar's rules.

Experiment 15: Probabilistic Context-Free Grammar Parsing in Python

1. Aim

To write a Python program to implement probabilistic context-free grammar (PCFG) parsing for a sentence.

2. Procedure

1. Import `PCFG` and `ViterbiParser` from the `nltk` module.
2. Define a probabilistic grammar with rule probabilities using `PCFG.fromstring()`.
3. Create a parser object using `nltk.ViterbiParser()`.
4. Parse the tokenized sentence to compute the most probable parse tree.
5. Display the resulting probabilistic parse tree and its probability score.

3. Program

```
import nltk
from nltk import PCFG

pcfg = PCFG.fromstring("S -> NP VP [1.0]\nNP -> Det N [0.6] | 'John' [0.4]\nVP -> V NP [1.0]\nDet -> 'the' [1.0]\nN -> 'dog' [1.0]\nV -> 'saw' [1.0]")

parser = nltk.ViterbiParser(pcfg)
sentence = "John saw the dog".split()

for tree in parser.parse(sentence):
    print("PCFG Parse:", tree)
```

4. Output

```
PCFG Parse: (S (NP John) (VP (V saw) (NP (Det the) (N dog)))) (p=0.24)
```

5. Result

Thus, the Python program was successfully executed to implement probabilistic context-free grammar parsing using `ViterbiParser`.

Experiment 16: Named Entity Recognition using SpaCy in Python

1. Aim

To write a Python program using the SpaCy library to perform Named Entity Recognition (NER) on a given text.

2. Procedure

1. Import the `spacy` library in Python.
2. Load the English language model using `spacy.load('en_core_web_sm')`.
3. Define a sample text containing named entities.
4. Process the text with `nlp(text)` and iterate over `doc.ents`.
5. Display each recognized named entity and its entity label.

3. Program

```
import spacy

nlp = spacy.load("en_core_web_sm")
text = "Google was founded in California in 1998."

doc = nlp(text)
entities = [(ent.text, ent.label_) for ent in doc.ents]

print("Named Entities:", entities)
```

4. Output

```
Named Entities: [('Google', 'ORG'), ('California', 'GPE'), ('1998',
'DATE')]
```

5. Result

Thus, the Python program was successfully executed to perform Named Entity Recognition (NER) on a given text using `spacy`.

Experiment 17: Accessing WordNet Synsets and Meanings in Python

1. Aim

To write a Python program that demonstrates how to access WordNet to retrieve synsets and explore word meanings.

2. Procedure

1. Import `wordnet` from the `nltk.corpus` module.
2. Use `wordnet.synsets()` to retrieve all synsets for a given word.
3. Access the first synset and extract its definition using `definition()`.
4. Extract the synonyms (lemma names) using `lemma_names()`.
5. Display the synset list, definition, and synonyms.

3. Program

```
from nltk.corpus import wordnet

syns = wordnet.synsets("program")
print("Synset Name:", syns[0].name())

print("Definition:", syns[0].definition())

print("Synonyms:", syns[0].lemma_names())
```

4. Output

```
Synset Name: plan.n.01
Definition: a series of steps to be carried out or goals to be
accomplished
Synonyms: ['plan', 'program', 'programme']
```

5. Result

Thus, the Python program was successfully executed to access WordNet, retrieve synsets, and explore word meanings using `wordnet.synsets()`.

Experiment 18: First-Order Predicate Calculus Parser in Python

1. Aim

To write a Python program to implement a simple First-Order Predicate Calculus (FOPC) parser for basic logical expressions.

2. Procedure

1. Import `Expression` from the `nltk.sem` module.
2. Initialize the parser function `Expression.fromstring`.
3. Define basic logical expressions using quantifiers, predicates, and operators.
4. Parse the logical strings into FOPC expression objects.
5. Display the parsed logical expressions.

3. Program

```
from nltk.sem import Expression

read_expr = Expression.fromstring

e1 = read_expr("man(socrates)")
print("Parsed Expression 1:", e1)

e2 = read_expr("all x.(man(x) -> mortal(x))")
print("Parsed Expression 2:", e2)
```

4. Output

```
Parsed Expression 1: man(socrates)
Parsed Expression 2: all x.(man(x) -> mortal(x))
```

5. Result

Thus, the Python program was successfully executed to implement a simple FOPC parser for logical expressions using `Expression.fromstring`.

Experiment 19: Word Sense Disambiguation using Lesk in Python

1. Aim

To write a Python program for word sense disambiguation using the Lesk algorithm.

2. Procedure

1. Import `lesk` from `nltk.wsd` and `word_tokenize` from `nltk.tokenize`.
2. Define two sentences containing an ambiguous word in different contexts.
3. Tokenize the sentences and apply `lesk()` on the target word.
4. Retrieve the disambiguated WordNet synset and its definition.
5. Display the predicted sense and definition for both sentences.

3. Program

```
from nltk.wsd import lesk
from nltk.tokenize import word_tokenize

s1 = lesk(word_tokenize("I deposited money in the bank"), "bank")
print("Sense 1:", s1.name(), "-", s1.definition())

s2 = lesk(word_tokenize("The boat was tied to the river bank"), "bank")
print("Sense 2:", s2.name(), "-", s2.definition())
```

4. Output

```
Sense 1: savings_bank.n.02 - a container for keeping money safely
Sense 2: bank.n.01 - sloping land (especially the slope beside a body of
water)
```

5. Result

Thus, the Python program was successfully executed to perform word sense disambiguation using the `lesk()` algorithm.

Experiment 20: Information Retrieval System using TF-IDF in Python

1. Aim

To write a Python program to implement a basic information retrieval system using TF-IDF for document ranking.

2. Procedure

1. Import `TfidfVectorizer` and `cosine_similarity` from `sklearn`.
2. Define a corpus of documents and a search query string.
3. Transform the documents and query into TF-IDF vectors using `TfidfVectorizer`.
4. Compute cosine similarity scores between the query and all documents.
5. Display the similarity scores and the highest-ranked document.

3. Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["Python is used for NLP.", "NLP processes human language.", "Java is a language."]
vec = TfidfVectorizer()

tfidf = vec.fit_transform(docs)
scores = cosine_similarity(vec.transform(["Python NLP"]), tfidf)[0]
print("Document Scores:", scores.round(2))
print("Top Ranked Document:", docs[scores.argmax()])
```

4. Output

```
Document Scores: [0.68 0.25 0.  ]
Top Ranked Document: Python is used for NLP.
```

5. Result

Thus, the Python program was successfully executed to implement a basic information retrieval system using TF-IDF for document ranking.

Experiment 21: Syntax-Driven Semantic Analysis in Python

1. Aim

To write a Python program that performs syntax-driven semantic analysis by extracting noun phrases and their meanings from a sentence.

2. Procedure

1. Import `nltk` and `wordnet` in Python.
2. Tokenize and POS-tag a sample input sentence.
3. Use `nltk.RegexpParser()` with grammar `NP: {<DT>?<JJ>*<NN>}` to extract noun phrases.
4. Retrieve the head noun of each extracted `NP` chunk and look up its WordNet meaning.
5. Display the extracted noun phrases and their semantic definitions.

3. Program

```
import nltk
from nltk.corpus import wordnet as wn

tagged = nltk.pos_tag(nltk.word_tokenize("The smart boy read a book"))
tree = nltk.RegexpParser("NP: {<DT>?<JJ>*<NN>}").parse(tagged)

for sub in tree.subtrees(lambda t: t.label() == "NP"):
    words = [w for w, t in sub.leaves()]
    print("NP:", " ".join(words), "| Meaning:", wn.synsets(words[-1])[0].definition())
```

4. Output

```
NP: The smart boy | Meaning: a youthful male person
NP: a book | Meaning: a written work or composition that has been
published
```

5. Result

Thus, the Python program was successfully executed to perform syntax-driven semantic analysis by extracting noun phrases and their meanings.

Experiment 22: Reference Resolution in Text using Python

1. Aim

To write a Python program that performs pronominal reference resolution within a text.

2. Procedure

1. Define a dictionary of entities and their gender attributes along with pronoun mappings.
2. Define a sample text containing named entities and pronouns.
3. Iterate through the words in the text and track the latest antecedent entity.
4. Resolve each pronoun to its matching antecedent noun.
5. Display the resolved pronoun-entity pairs.

3. Program

```
entities = {"John": "M", "Mary": "F"}
pronouns = {"he": "M", "she": "F"}

text = "John met Mary and she greeted him while he smiled"
context = {}

for w in text.split():
    if w in entities: context[entities[w]] = w
    elif w in pronouns: print(f"Pronoun '{w}' refers to:", context[pronouns[w]])
```

4. Output

```
Pronoun 'she' refers to: Mary
Pronoun 'he' refers to: John
```

5. Result

Thus, the Python program was successfully executed to perform reference resolution of pronouns within a text.

Experiment 23: Evaluating Text Coherence in Python

1. Aim

To write a Python program that evaluates the coherence of a given text.

2. Procedure

1. Import `TfidfVectorizer` and `cosine_similarity` from `sklearn`.
2. Define a list of consecutive sentences from a text passage.
3. Convert the sentences into TF-IDF vectors using `fit_transform()`.
4. Calculate the cosine similarity between adjacent sentences.
5. Display the adjacent similarity scores and the mean coherence score.

3. Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

sents = ["NLP is a subfield of AI.", "AI helps computers understand language.", "Computers process language data."]
vecs = TfidfVectorizer().fit_transform(sents)

s1 = cosine_similarity(vecs[0], vecs[1])[0][0]
s2 = cosine_similarity(vecs[1], vecs[2])[0][0]
print("Pairwise Similarities:", round(s1, 2), round(s2, 2))
print("Text Coherence Score:", round((s1 + s2) / 2, 2))
```

4. Output

```
Pairwise Similarities: 0.22 0.41
Text Coherence Score: 0.32
```

5. Result

Thus, the Python program was successfully executed to evaluate the coherence of a given text using sentence similarity.

Experiment 24: Dialog Act Recognition in Python

1. Aim

To write a Python program that recognizes dialog acts in a given dialog or conversation.

2. Procedure

1. Define a function to classify conversational utterances into dialog act categories.
2. Check for question marks, greeting words, and farewell phrases in each utterance.
3. Define a sample conversation containing multiple turns.
4. Apply the dialog act recognition function to each utterance.
5. Display each utterance along with its recognized dialog act.

3. Program

```
def get_dialog_act(text):
    if text.endswith("?"): return "Question"
    if text.lower().startswith(("hi", "hello")): return "Greeting"
    if text.lower().startswith(("bye", "thanks")): return "Closing"
    return "Statement"

dialog = ["Hello!", "How are you?", "I am learning NLP.", "Thanks, bye!"]
for turn in dialog:
    print(f"{turn} -> {get_dialog_act(turn)}")
```

4. Output

```
Hello! -> Greeting
How are you? -> Question
I am learning NLP. -> Statement
Thanks, bye! -> Closing
```

5. Result

Thus, the Python program was successfully executed to recognize dialog acts in a given conversation.

Experiment 25: Text Generation using OpenAI GPT-3 in Python

1. Aim

To write a Python program that utilizes the GPT-3 model to generate text based on a given prompt using the OpenAI library.

2. Procedure

1. Import the `openai` library in Python.
2. Set the OpenAI API key for authentication.
3. Define a text prompt for the GPT-3 model.
4. Use `openai.Completion.create()` with `text-davinci-003` to generate text.
5. Display the input prompt and the generated text output.

3. Program

```
import openai

openai.api_key = "YOUR_API_KEY"
prompt = "Write a short definition of Artificial Intelligence."

response = openai.Completion.create(engine="text-davinci-003", prompt=prompt, max_tokens=40)

print("Prompt:", prompt)
print("Generated Text:", response.choices[0].text.strip())
```

4. Output

```
Prompt: Write a short definition of Artificial Intelligence.
Generated Text: Artificial Intelligence is the simulation of human
intelligence processes by computer systems.
```

5. Result

Thus, the Python program was successfully executed to generate text based on a given prompt using the OpenAI GPT-3 library.

Experiment 26: Machine Translation using Transformers in Python

1. Aim

To write a Python program to implement machine translation from English to French using the Hugging Face Transformers library.

2. Procedure

1. Import `pipeline` from the `transformers` library.
2. Initialize the translation pipeline using `pipeline('translation_en_to_fr')`.
3. Define a sample English sentence to translate.
4. Pass the English text to the translator pipeline.
5. Display the English input and the translated French text.

3. Program

```
from transformers import pipeline

translator = pipeline("translation_en_to_fr")
text = "Python is easy to learn and very useful."

output = translator(text)
print("English:", text)
print("French:", output[0]['translation_text'])
```

4. Output

```
English: Python is easy to learn and very useful.
French: Python est facile à apprendre et très utile.
```

5. Result

Thus, the Python program was successfully executed to translate English text to French using the Hugging Face `transformers` library.